



Transpiladores

Programación con JavaScript II



Actividad de Bienestar





Temas

- | | |
|---|---|
| 1 | Manejadores de paquetes y empaquetados de módulos |
| 2 | Nuevos tipos y características |
| 3 | Asincronía |
| 4 | API |
| 5 | Ajax |
| 6 | Scope y This |
| 7 | Transpiladores |
| 8 | Pruebas |





Prework

- Leer detenidamente y comprender el material explicado en el tema **Transpiladores**.
- Practicar todos los ejemplos que se describen previamente en el tema **Transpiladores**.
- Revisar cada uno de los recursos adicionales que se proponen en el tema **Transpiladores**.



Para saber más

Lecturas

- Kantor, I. (2022). Polyfill y transpiladores. Recuperado de <https://es.javascript.info/polyfills>
- Riyaa7vermaa. (2022). How to use polyfill in JavaScript? Recuperado de <https://www.geeksforgeeks.org/how-to-use-polyfill-in-javascript/>



Para saber más

Videos

- Midulive. (2022, 22 de mayo). ¿Qué es un Polyfill? ¿Cómo crear un polyfill desde cero? Típica pregunta de entrevista  [Archivo de video]. Recuperado de <https://www.youtube.com/watch?v=DyfX6ZWVky4>.
- Steve Griffith - Prof3ssorSt3v3. (2018, 20 de julio). Creating JavaScript Polyfills [Archivo de video]. Recuperado de <https://www.youtube.com/watch?v=qTCM8td2klQ>



Actividad Guiada

Parte 1

Duración: 75 minutos.



Ejercicio 1



Objetivo:

Comprender y aplicar la transpilación de JavaScript moderno mediante Babel, identificando su utilidad para la compatibilidad entre versiones y navegadores.

- ◆ Comprender qué es un transpilador y el problema de compatibilidad que resuelve.
- ◆ Reconocer la utilidad de Babel y compararlo con otras herramientas de transpilación.
- ◆ Analizar y explicar el funcionamiento de código JavaScript moderno.
- ◆ Transformar código utilizando Babel para hacerlo compatible con versiones anteriores de JavaScript.
- ◆ Diferenciar el propósito de un transpilador y un Polyfill.
- ◆ Investigar la compatibilidad de características de ECMAScript con diferentes navegadores.



Apóyate de una IA para investigar y construye tu propia interpretación en el Padlet.



Transpilador & Babel



Investiga y explica:

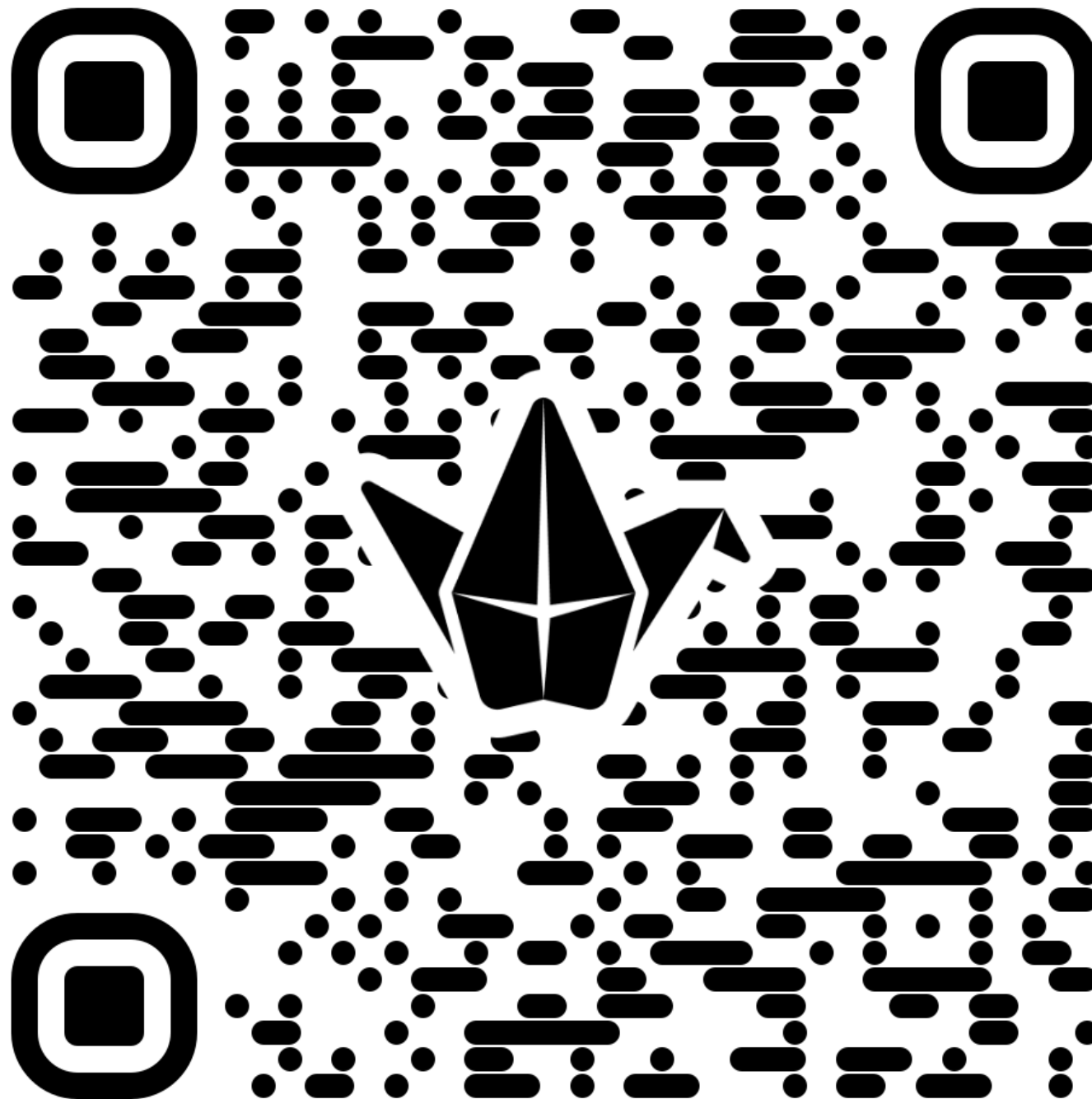
1. ¿Qué es un transpilador y por qué es útil en el desarrollo web con JS?
2. ¿Cómo le explicarías a un compañero programador la utilidad de Babel?
3. Ventajas de usar Babel frente a otros transpiladores.
4. Justificación del uso de código Polyfill en un proyecto.



Apóyate de una IA y escribe tu interpretación en el Padlet.



Padlet





¿Qué es un transpilador y por qué es útil en el desarrollo web con JS?

Un **transpilador** es una **herramienta** que **convierte** **código fuente** de un lenguaje a otro lenguaje **de similar nivel**. En el caso de JavaScript, transforma **código moderno (ES6+)** en una **versión anterior** (como ES5), que es **compatible con más navegadores**.

¿Por qué es útil?

Porque **permite escribir código moderno**, limpio y con nuevas funcionalidades **sin preocuparse** por si los **navegadores antiguos** lo van a entender. El transpilador se encarga de convertirlo para asegurar compatibilidad.



¿Cómo le explicarías a un compañero programador la utilidad de Babel?

Babel es una **herramienta** que te permite usar lo último de JavaScript hoy, sin tener que esperar a que todos los navegadores lo soporten. Tú escribes con **let, const, arrow functions, clases, etc.**, y Babel lo convierte a una **versión más vieja** de JS que los navegadores entienden sin problema. Así puedes desarrollar sin restricciones y asegurar que tu app funciona para todos.



Ventajas de usar Babel frente a otros transpiladores

 **Amplio soporte de características de JavaScript moderno** (ES6, ES7, etc.).

 **Sistema de plugins** personalizable. Puedes elegir qué funcionalidades transformar.

 **Gran comunidad y documentación**, lo que facilita encontrar soporte y ejemplos.

 **Integración sencilla** con herramientas como Webpack, Parcel o Vite.

 **Compatible con polyfills**, usando @babel/preset-env y core-js.



Justificación del uso de código Polyfill en un proyecto

Un **polyfill** es un **fragmento de código** (generalmente JS) que **simula una funcionalidad moderna en navegadores que no la soportan**.

¿Por qué usarlo?

Porque asegura que **todas las funcionalidades modernas** (como `Promise`, `Array.prototype.includes`, `fetch`, etc.) estén disponibles incluso en navegadores más antiguos. Así, tu aplicación será **más robusta y accesible** para todos los usuarios.

 Ejemplo: si usas `Promise` y un navegador no lo soporta, puedes usar un polyfill como `core-js` para que ese navegador se comporte como si lo tuviera.

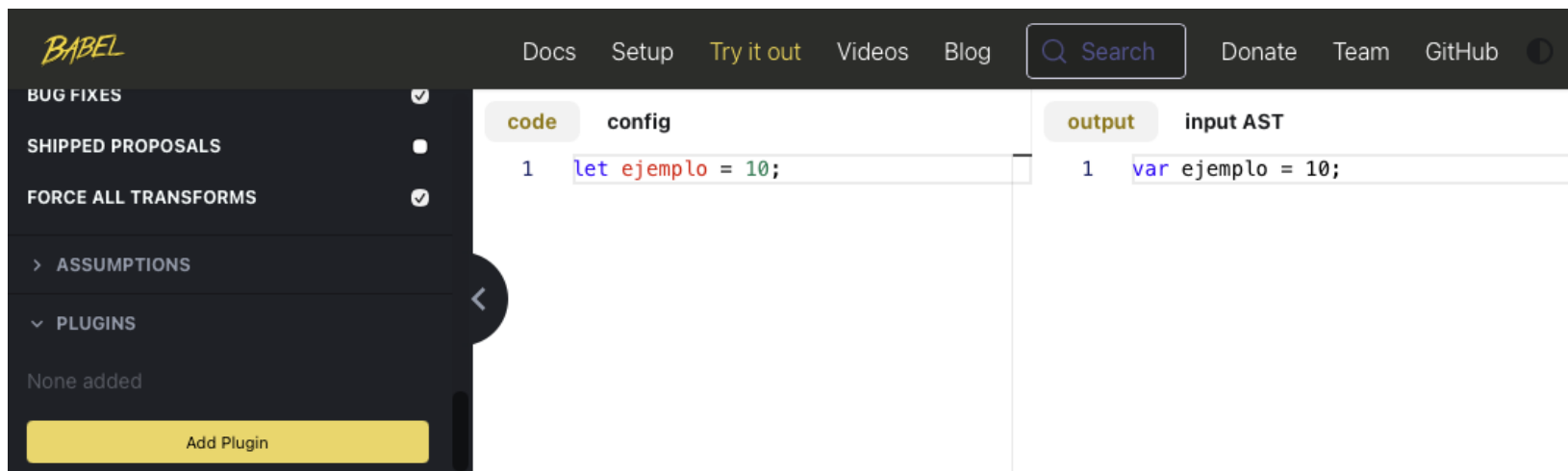


Ejercicio 1: Uso de Babel



Instrucciones:

- Analiza la tabla con fragmentos de código.
- Agrega en la segunda columna el código transformado



The screenshot shows the Babel REPL interface. On the left is a sidebar with navigation links: BABEL, BUG FIXES (checked), SHIPPED PROPOSALS, FORCE ALL TRANSFORMS (checked), > ASSUMPTIONS, < PLUGINS, and None added. At the bottom of the sidebar is a yellow button labeled 'Add Plugin'. The main area has a top navigation bar with links: Docs, Setup, Try it out, Videos, Blog, a search bar, Donate, Team, and GitHub. Below the navigation bar are two tabs: 'code' and 'config'. The 'code' tab is active, showing a text input with the code '1 let ejemplo = 10;'. To the right of the input is a 'config' tab. Further right are two output tabs: 'output' and 'input AST'. The 'output' tab is active, showing the transformed code '1 var ejemplo = 10;'. The 'input AST' tab is empty.



Transpilación con Babel



Paso a paso:

- Copia cada fragmento de código.
- Pega en babeljs.io/repl.
- Observa y copia el código convertido a versiones compatibles.

Verifica cómo Babel transforma sintaxis moderna a versiones anteriores.



Códigos

```
var array = [1, 2, 3, 4, 5];
```

```
var getRandomNumber = function getRandomNumber  
( ) {  
    return Math.ceil(Math.random() * 6);  
};
```

```
function add() {  
    var a = arguments.length > 0 && arguments[0]  
        !== undefined ? arguments[0] : 5;  
    var b = arguments.length > 1 && arguments[1]  
        !== undefined ? arguments[1] : 10;  
    return a + b;  
}  
console.log(add());
```



Códigos

```
var add2 = function add(a, b) {  
  return a + b;  
};
```

```
function* hola(){  
  yield "Hola Gabriela";  
  yield "Hola Rocio";  
  yield "Hola Andrea";  
}
```

```
var helloInstance = hola();  
console.log(helloInstance.next());  
console.log(helloInstance.next());  
console.log(helloInstance.next());  
console.log(helloInstance.next());
```



ECMAScript 2022



Investiga:

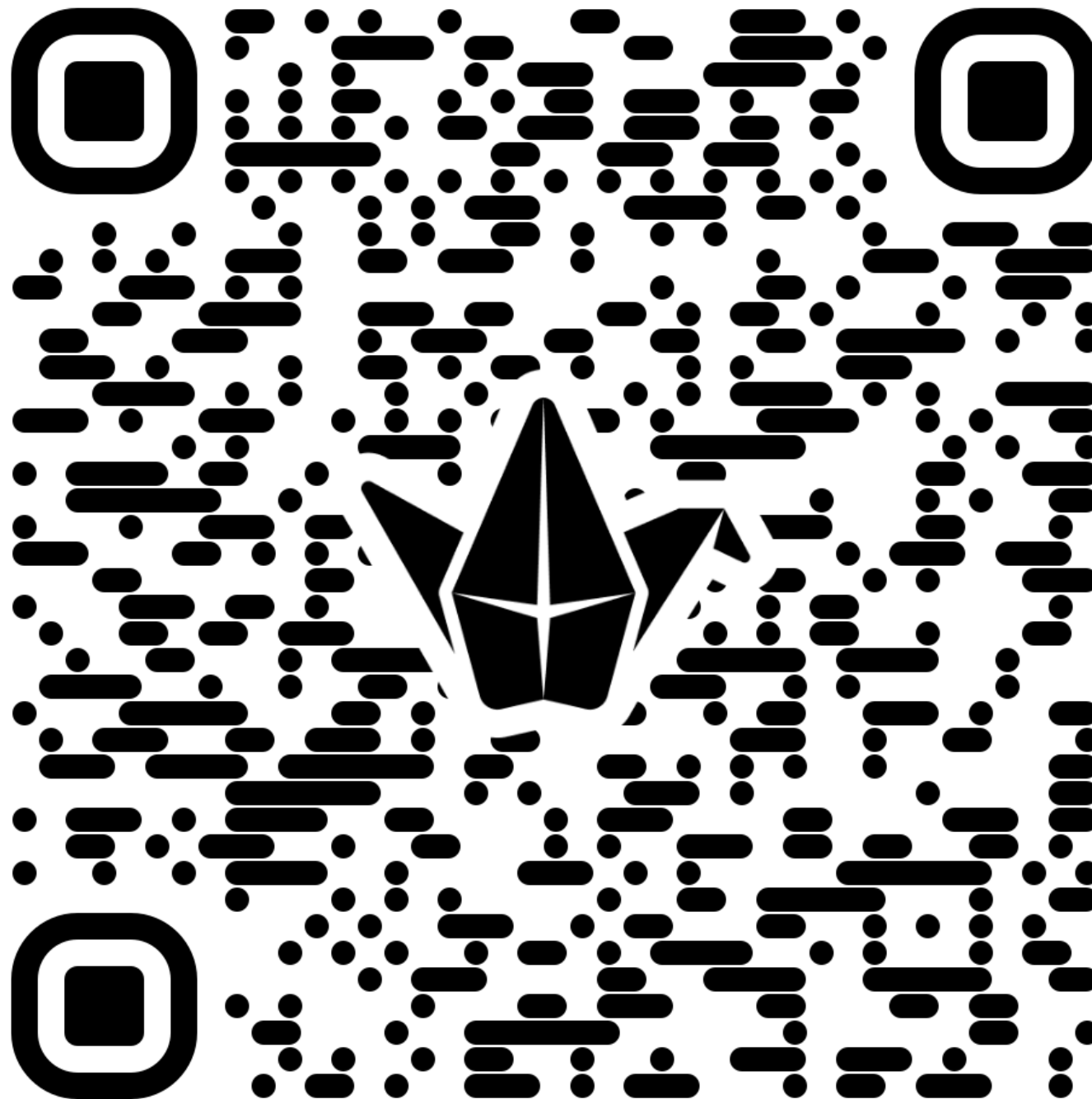
- Tres funciones nuevas de ECMAScript 2022.
- ¿Qué navegadores las soportan?
- ¿Requieren transpilación o polyfill?



Apóyate de una IA, busca ejemplos reales de uso y escribe tu interpretación en el Padlet.



Padlet





Array.prototype.at()

Permite acceder a elementos de un array usando índices positivos o negativos (como en Python).

```
const arr = [10, 20, 30];  
console.log(arr.at(-1)); // 30
```



Soporte de navegadores:



Chrome 92+, Firefox 90+, Edge 92+, Safari 15+



No disponible en versiones antiguas o IE



¿Requiere transpilación o polyfill?



Polyfill: Sí, si necesitas soportar navegadores antiguos.



Transpilación: No posible (no se puede transpilar acceso de índices negativos de forma automática).



Object.hasOwn()

Es una alternativa a `Object.prototype.hasOwnProperty`, pero más segura y clara.

```
const obj = { a: 1 };  
console.log(Object.hasOwn(obj, 'a')); // true
```



Soporte de navegadores:



Chrome 93+, Firefox 92+, Edge 93+, Safari 15.4+



No soportado en navegadores antiguos



¿Requiere transpilación o polyfill?



Polyfill: Sí (puedes definirlo tú mismo si no existe).



Transpilación: No se transpila automáticamente con Babel, necesita fallback manual o polyfill.



Class Fields (propiedades públicas y privadas en clases)

Permite definir propiedades directamente en la clase, incluyendo privadas con #.

```
class Person {  
  #secret = '123';  
  name = 'Ana';  
  
  getSecret() {  
    return this.#secret;  
  }  
}
```



Soporte de navegadores:



Chrome 72+ (privadas desde 84), Firefox 90+, Edge 84+, Safari 13.1+



¿Requiere transpilación o polyfill?



Transpilación: Sí, para soporte total de campos privados (Babel puede hacerlo).



Polyfill: No aplican bien para campos privados (requieren cambios estructurales, no solo funciones).



Función	¿Requiere Transpilación?	¿Requiere Polyfill?	Soporte común
Array.at()	 No	 Sí	Bueno desde 2021
Object.hasOwn()	 No	 Sí	Bueno desde 2021
Class Fields	 Sí (en algunos casos)	 No (difícil de polyfillear)	Bueno desde 2020–2021



Ejercicio 2

Objetivo:

Analizar la compatibilidad de código JavaScript moderno y aplicar herramientas de transpilación y Polyfills para facilitar su ejecución en distintos entornos.

- ◆ Analizar el funcionamiento de código que utiliza Promise, async y await.
- ◆ Identificar características que pueden presentar problemas de compatibilidad.
- ◆ Aplicar Babel para transformar código JavaScript moderno.
- ◆ Comprender el propósito de los Polyfills en la compatibilidad de aplicaciones web.
- ◆ Comprobar el funcionamiento del código transformado en un navegador.



Análisis de Código y Polyfill



Actividades:

- Analiza el código proporcionado y explica qué hace.
- Determina cuál función necesita convertirse en Polyfill y justifica por qué.
- Usa Babel para convertir esa función en un Polyfill.
- Integra el código en un archivo .html y comprueba que funcione correctamente.



Piensa en la compatibilidad con navegadores antiguos.



Analiza el código y explica qué hace

```
function responderDespuesde2sec() {  
  return new Promise(resolver => {  
    setTimeout(() => {  
      resolver('respuesta');  
    }, 2000);  
  });  
}  
  
async function asyncCall() {  
  console.log('llamando');  
  const resultado = await responderDespuesde2sec();  
  console.log(resultado);  
}  
  
asyncCall();
```



¿Qué necesita Polyfill?

Las siguientes **características modernas de JavaScript** están en uso:

Característica	¿Soporte amplio?	¿Requiere polyfill/transpilación?
Promise	No en IE11 o muy antiguos	 Polyfill necesario
async/await	No soportado en IE11	 Requiere transpilación con Babel
setTimeout	Soportado desde siempre	 No necesita polyfill



Convertir con Babel (ES6 a ES5)

Babel **no puede polyfillar automáticamente Promesas**, pero puede transpilar `async/await` a código que usa Promesas y callbacks.



Receso

Duración: 10 minutos.



Actividad Guiada

Parte 2

Duración: 75 minutos.



Ejercicio 3

Objetivo:

Analizar la compatibilidad de diferentes características de JavaScript y seleccionar estrategias para facilitar su ejecución en navegadores con distintas capacidades.

- ◆ Identificar características de JavaScript que presentan limitaciones de compatibilidad.
- ◆ Consultar la compatibilidad de funciones y métodos en distintos navegadores.
- ◆ Proponer alternativas para ejecutar código en entornos que no soportan determinadas características.
- ◆ Aplicar técnicas de detección de funcionalidades del navegador.
- ◆ Evaluar la utilidad y las limitaciones de Babel como herramienta de compatibilidad.



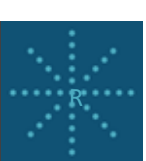
Analiza el código

```
// Part 1  
var str = 'Copa de Futbol';  
  
console.log(str.includes('Copa')); // true  
console.log(str.includes('Torneo')); // false
```



Parte 1: Preguntas de reflexión

- ¿Cuál sería el resultado esperado en la parte 1 en la consola?
- ¿Es compatible `str.includes()` con IE11?
- ¿Qué alternativa usarías si no lo fuera?



◆ Parte 1

```
var str = 'Copa de Futbol';
```

```
console.log(str.includes('Copa'));    // true  
console.log(str.includes('Torneo'));  // false
```

✅ `str.includes('Copa')` devuelve true porque **"Copa"** está dentro del string.

✅ `str.includes('Torneo')` devuelve false porque **no está** en el string.

📌 `includes()` realiza una **búsqueda exacta** y distingue mayúsculas/minúsculas.



Compatibilidad de `str.includes()` con IE11



`String.prototype.includes()` no es compatible con Internet Explorer 11.

Aunque funciona en navegadores modernos, IE11 no lo soporta de forma nativa.



Alternativa para includes() en IE11

Usa indexOf() como reemplazo funcional:

```
console.log(str.indexOf('Copa') !== -1); // true  
console.log(str.indexOf('Torneo') !== -1); // false
```

 **indexOf()** retorna la posición del substring si lo encuentra, o **-1** si no. Por eso comparamos con **!== -1** para emular el comportamiento de **includes()**.



Analiza el código

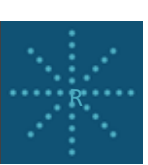
```
// Part 2
class miClase {
  constructor(a, b) {
    this.result = a + b;
  }
}

const miFuncion = (a, b) => a + b;
const resultado = [1, 2].reduce((a, b) => a + b, 0);
```



Parte 2:

- Explica el funcionamiento del código de la Parte 2.
- Identifica qué parte no es compatible con IE11.
- Propón una solución alternativa que sí sea compatible.



◆ Parte 2: Clase y funciones flecha

```
class miClase {  
  constructor(a, b) {  
    this.result = a + b;  
  }  
}
```

✓ class define una clase moderna (ES6), con un constructor que suma dos valores.

```
const miFuncion = (a, b) => a + b;
```

✓ miFuncion es una **función flecha**, equivalente a `function(a, b) { return a + b; }`

```
const resultado = [1, 2].reduce((a, b) => a + b, 0);
```

✓ `reduce()` recorre el array `[1, 2]` y suma sus elementos, iniciando en 0. Resultado: 3



Característica	Compatible con IE11	Requiere transpilación
class	 No	 Sí (con Babel)
Arrow functions	 No	 Sí (con Babel)
Array.prototype.reduce()	 Sí	 No



Propuesta de alternativa compatible IE11

```
// Simulación de clase usando función constructora
function miClase(a, b) {
  this.result = a + b;
}

// Función tradicional (en lugar de arrow function)
function miFuncion(a, b) {
  return a + b;
}

// reduce() sí funciona en IE11
var resultado = [1, 2].reduce(function (a, b) {
  return a + b;
}, 0);

console.log("Resultado con reduce:", resultado);

// Probando la clase
var instancia = new miClase(5, 7);
console.log("Resultado en miClase:", instancia.result);
```



Parte 3: Evaluación y Reflexión



Responde:

- ¿Qué sentencia `if()` usarías para detectar falta de soporte?
- ¿Te parece útil Babel para este propósito?
- ¿Qué desventajas tiene usar Babel?



Reflexión crítica sobre herramientas modernas y decisiones técnicas en proyectos reales.



¿Qué sentencia if() usarías para detectar falta de soporte?

```
// Verificar si NO existe Promise (indicador de JS moderno)
if (typeof Promise === 'undefined') {
    alert('Este navegador no soporta Promesas. Considera usar un
polyfill.');
```

```
}

// Verificar si no soporta Array.prototype.includes
if (!String.prototype.includes) {
    alert('El método includes no está soportado. Puedes usar
indexOf o un polyfill.');
```



¿Te parece útil Babel para este propósito?

Sí, Babel es una herramienta muy útil y práctica.

- Convierte **código moderno (ES6+)** a **ES5**, compatible con navegadores antiguos como IE11.
- Permite que los desarrolladores usen las **últimas funciones de JavaScript** (clases, async/await, arrow functions, etc.) sin preocuparse por la compatibilidad.
- Se integra fácilmente con herramientas como **Parcel, Webpack o Vite**.
- Junto con core-js, Babel también puede manejar **polyfills automáticos** con @babel/preset-env.



Desventaja de USAR Babel	Explicación
 Tamaño del bundle más grande	Al transpilar y agregar polyfills, el archivo final puede crecer bastante.
 Rendimiento ligeramente menor	El código transpileado no siempre es tan eficiente como el nativo.
 Necesita configuración	Requiere instalación, presets, y a veces ajustes según el proyecto.
 No puede transpilar todo	Babel transpila sintaxis, pero no siempre puede reemplazar APIs faltantes , como fetch o Intl. Para eso se necesita un polyfill adicional .
 Depuración más difícil	El código que ves en el navegador no es exactamente el que escribiste, lo que puede complicar el debugging (aunque el source map ayuda).



Cierre

Duración: 10 minutos.



¿Qué aprendimos hoy?



Transpilación
Transformar código JavaScript moderno para facilitar su compatibilidad con otros entornos.



Babel
Utilizar Babel para transformar características modernas de JavaScript y analizar su utilidad.



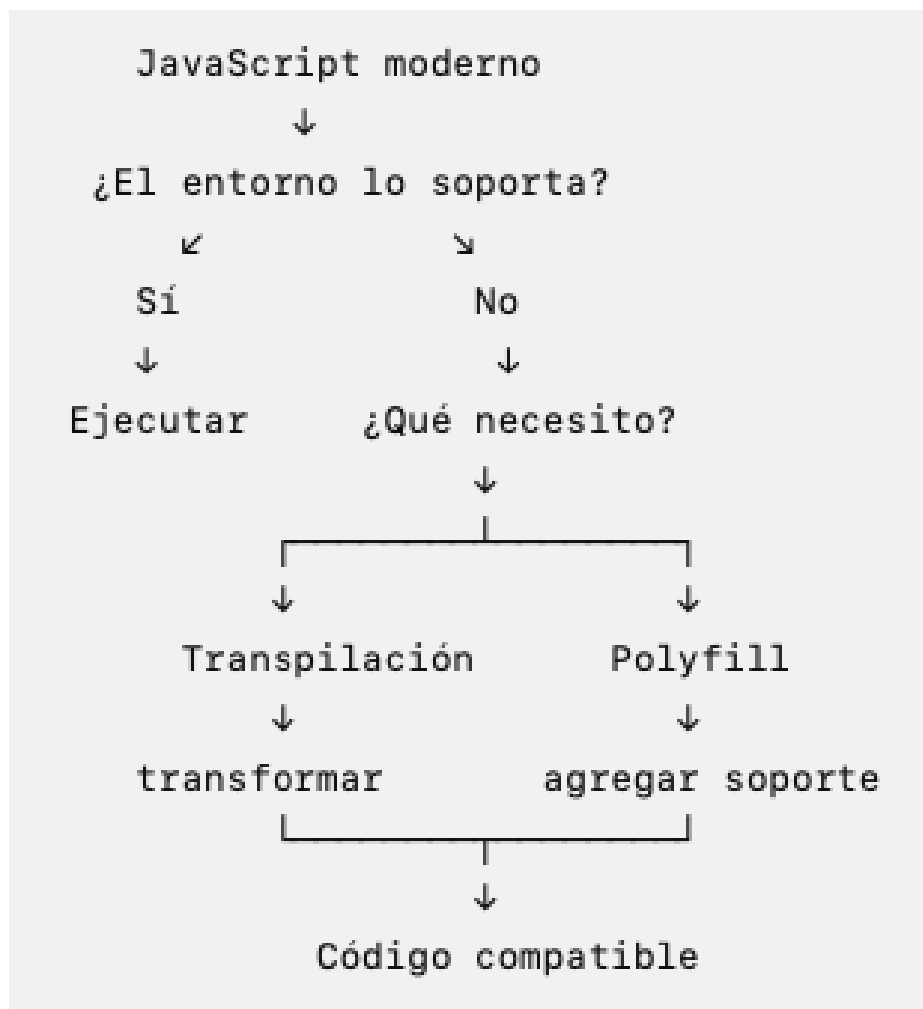
Polyfills
Comprender cuándo una funcionalidad requiere una implementación adicional para ser utilizada en entornos que no la soportan.



Compatibilidad
Analizar las capacidades de los navegadores y elegir estrategias para ejecutar nuestro código de forma compatible.



De JavaScript moderno a código compatible



Desarrollar para la web no solo implica escribir código moderno, sino considerar dónde y cómo se ejecutará.